S.No: 10	Exp. Name: ***Depth-First Search (DFS)***	Date:2026-04-12

Aim:

Write a C program to implement and perform a Depth-First Search (DFS) on a directed graph represented using an adjacency list. The program should print all vertices reachable from a given starting vertex in the order they are visited by DFS.

Input Format:

- The first line contains two integers: V (number of vertices) and E (number of edges).
- The next E lines each contain two integers: u and v, where there is a directed edge from node u to node v.
- The last line contains a single integer, which is the starting node for the DFS traversal.

Output Format:

- Print the vertices reachable from the starting vertex in the exact order they are visited by DFS.
- Each vertex should be printed, followed by a single space.
- The output should be printed on one line only as:

```
v1 v2 v3 ... vn
```

Constraints:

- $1 \leq V \leq 100$
- $0 \leq E \leq V \times (V-1)$
- Vertices are numbered from 0 to $\mathrm{V}-1$

Note:

- In DFS, the smaller numbered node is visited first when there are multiple adjacent nodes, ensuring a consistent traversal order.
- Refer to the visible test cases for a better understanding.

Source Code:

dfs_directed_graph.c

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTICES 100

// Graph represented using adjacency list
typedef struct Node {
    int vertex;
    struct Node* next;
} Node;

Node* adjList[MAX_VERTICES];
int visited[MAX_VERTICES];

// Function to create a new adjacency list node
Node* createNode(int vertex) {
        Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->vertex = vertex;
```

```
    newNode->next = NULL;
    return newNode;
}

// Function to add an edge to the graph
void addEdge(int u, int v) {
        Node* newNode = createNode(v);

    if (adjList[u] == NULL) {
        adjList[u] = newNode;
    } else {
        Node* temp = adjList[u];
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        }


    }

// Function to sort the adjacency list for each vertex
void sortAdjList(int V) {
    //write your code here...




}

// Depth-First Search (DFS) function
void DFS(int start) {
    //write your code here...
    visited[start] = 1;
    printf("%d ", start);

    Node* temp = adjList[start];
    while (temp != NULL) {
        if (!visited[temp->vertex]) {
            DFS(temp->vertex);
        }
        temp = temp->next;

        }
}

int main() {
    int V, E;
    int u, v, start;

    // Read number of vertices and edges
    scanf("%d %d", &V, &E);

    // Initialize adjacency list
    for (int i = 0; i < V; i++) {
        adjList[i] = NULL;
```

```
        visited[i] = 0;
    }

    // Read the edges
    for (int i = 0; i < E; i++) {
        scanf("%d %d", &u, &v);
        addEdge(u, v);
    }

    // Sort the adjacency list for each vertex
    sortAdjList(V);

    // Read the starting node
    scanf("%d", &start);

    // Perform DFS starting from the given node
    DFS(start);

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5 6
0 1
0 2
1 3
1 4
2 4
3 4
0
0 1 3 4 2

Test Case - 2
User Output
4 5
0 1
0 2
2 3
1 3
3 0
0
0 1 3 2

ID: 25070521290